Guía técnica

Arquitectura del Frontend, Login con Google, JWT y conexión con la base de datos

Proyecto: AppoinTech — agendamiento de reparación de electrodomésticos a domicilio
Stack: Angular 21 (NgModules) · Spring Boot · PostgreSQL

1. Arquitectura general

Bootstrap: `main.ts` → `platformBrowser().bootstrapModule(AppModule)`. El proyecto usa NgModules clásicos: todo componente declara `standalone: false` explícitamente, sin bootstrap standalone.

Capas del proyecto:

- **`core/`** — singletons sin UI: *services/* (lógica de negocio y llamadas HTTP), *guards/* (protección de rutas), *interceptors/* (middleware HTTP), *config/* (`API_BASE_URL`, `GOOGLE_CLIENT_ID`), *models/* (tipos compartidos como `RespuestaEnvuelta<T>`)
- **`shared/`** — componentes visuales reutilizables, agrupados en `SharedModule`
- **`features/`** — un módulo **lazy-loaded** por pantalla o flujo: `auth/`, `admin/`, `completar-perfil/`, `en-construccion/`

Ruteo: cada feature tiene su propio `*-routing.module.ts`. Las rutas protegidas usan guards funcionales (`CanActivateFn`): `adminGuard` exige rol ADMINISTRADOR, `authGuard` solo exige sesión activa.

2. Login con Google — flujo completo en el frontend

1

Carga del SDK

`index.html` incluye `<script src="https://accounts.google.com/gsi/client" async defer>`. Como carga de forma asíncrona, `GoogleLoginButtonComponent` hace polling hasta que `window.google.accounts.id` exista.

2 **Inicialización**

`google.accounts.id.initialize({ client_id: GOOGLE_CLIENT_ID, callback })` registra un callback, y `google.accounts.id.renderButton(...)` dibuja el botón oficial de Google dentro de un `<div>`.

3 **El usuario inicia sesión con Google**

El usuario hace clic y completa el flujo de Google (fuera de nuestra app). Google invoca nuestro callback con `{ credential: idToken }` — ese `idToken` es un **JWT firmado por Google**, no por nuestro backend.

4 **Zoneless: un detalle técnico no obvio**

Ese callback lo dispara el SDK de Google **fuera del ciclo de Angular** (esta app es zoneless, sin `zone.js`). Por eso el callback está envuelto en `NgZone.run()` + `ChangeDetectorRef.markForCheck()` — si no, la interfaz no se actualizaría tras la respuesta.

5 **Reenvío a nuestro backend**

El frontend reenvía ese token: `AuthService.loginConGoogle(idToken)` → `POST /api/auth/google { idToken }`.

6 **Verificación y emisión del JWT propio**

El backend (Spring Boot, repositorio separado) valida la firma de ese token contra las claves públicas de Google, busca o crea el usuario en la base de datos, y genera **nuestro propio JWT** de aplicación.

7 **Respuesta envuelta**

La respuesta llega como `{ exito, datos: { token, perfilCompleto }, mensaje, timestamp }`. El frontend hace `map(r => r.datos)` y guarda `datos.token`.

8 **Redirección según perfil y rol**

Según `perfilCompleto`, `redirigirSegunRolYPerfil()` manda a `/completar-perfil` o a la ruta correspondiente al rol del usuario.

3. El JWT en el frontend

- **Almacenamiento:** se guarda en `localStorage` bajo la clave `appointech_token` (sobrevive recargas y pestañas).
- **Nunca se valida criptográficamente en el cliente** — solo se decodifica: `decodificarToken()` parte el string por `"."`, hace `atob()` del segmento del payload y lo parsea como JSON. Es lectura de datos, no verificación de firma (eso ya lo hizo el backend al emitirlo).
- `estaAutenticado()` compara el claim `exp` contra la hora actual.
- `obtenerRol()` lee el claim `rol` directamente del token — **el rol nunca viaja en el body de la respuesta**, solo dentro del JWT.
- **Adjunto automático:** `authInterceptor` (un `HttpInterceptorFn` registrado en `AppModule`) clona cada `HttpRequest` saliente agregando `Authorization: Bearer <token>`. Ningún servicio lo adjunta manualmente.
- **Cerrar sesión** es solo borrar la entrada de `localStorage` — no hay llamada al backend para invalidar el token (los JWT son stateless, expiran solos por `exp`).

4. Dónde entra la base de datos

El frontend **nunca toca la base de datos directamente**. Todo pasa por `API_BASE_URL` (`core/config/api.config.ts` → `http://localhost:8080/api`), que apunta al backend Spring Boot.

A partir de ahí, todo lo de persistencia (Controller → Service → Repository → PostgreSQL) vive en el otro repositorio, fuera de este código.

Desde la perspectiva del frontend, el **límite del sistema** es exactamente esa constante: todo lo que cruza la frontera HTTP es responsabilidad del backend.

```
Usuario → Frontend (Angular) → API_BASE_URL / HTTP → Backend (Spring Boot) →
Base de datos (PostgreSQL)
```